

Universidad de Guadalajara



Centro universitario de ciencias exactas e ingenierías

*DIVISIÓN DE TECNOLOGÍAS PARA LA INTEGRACIÓN CIBER-
HUMANA*

INGENIERÍA EN COMPUTACIÓN

***Departamento de Innovación Basada en la Información y el
Conocimiento***

Reporte de actividades

Prestador: Mancillas Magdaleno Erick Alberto

Receptor: Dr. Jorge de Jesús Gálvez Rodríguez

Nota importante: Este documento no tiene como objetivo ser una representación científica o matemática correcta.

ÍNDICE

Semana 1	4
Investigación	4
Semana 2	5
¿Qué es un algoritmo metaheurístico?	5
¿Qué es la optimización por enjambre de partículas (PSO)?	5
Ejemplo de PSO	6
Conclusiones	10
Semana 4 - 5	11
Introducción	11
Código en matlab	13
Conclusiones	19
Semana 6 -7	20
Evolución diferencial	20
¿Qué es el vuelo de Weibull?	20
Código en matlab	21
Resultados	25
Conclusiones	26
Semana 8 - 9	27
Introducción	24
Conceptualización del algoritmo TSA	28
Motivación teórica para el uso del enfoque tangente	28
Modelo matemático del algoritmo	28
Descripción del algoritmo	28
Arquitectura general del TSA (seudocódigo)	30
Conclusiones	34
Semana 10	35
Introducción	35
Resumen	36
Conversión a código	36
Implementación en DA (código completo)	39
Conclusiones	39

Semana 11 - 12.....	40
Comparación de resultados	40
Evaluación con benchmarks y condiciones especiales.....	42
Conclusiones	43
Semana 13 -14.....	44
Introducción	44
Funciones y archivos a considerar	45
Función principal.....	46
Ejemplo práctico	47
Semana 15 - 16.....	50
Introducción	50
Inicialización y Evaluación	50
Agrupamiento Jerárquico	51
Cálculo de Densidad y Centroides	52
Refinamiento Local	52
Control de fases y elitismo	54
Resultados	54
Sphere	55
Rastrigin	56
Sphere + Rastrigin.....	57
Conclusiones	57

Semana 1

Objetivos:

- Investigar y entender el concepto de optimización dentro del contexto de algoritmia.
- Investigar y entender el concepto de algoritmo metaheurístico dentro de la solución de problemas de optimización.

Desarrollo:

Investigación

La *algoritmia* utiliza métodos y técnicas para estudiar los algoritmos, sus propiedades y eficiencia. Además del desarrollo de métodos y técnicas para diseñar algoritmos y estructuras de datos. Por medio de conceptos y técnicas ya conocidas, se estudia nuevas técnicas de diseño algorítmico como el método voraz, la programación dinámica, entre otros. Cada una de estas técnicas se ilustra con ejemplos prácticos, muchos de ellos algoritmos y EDs de gran relevancia, como el algoritmo de Dijkstra para el cálculo de caminos mínimos en un grafo, el algoritmo de cálculo de la distancia de edición entre dos cadenas de caracteres, la prueba de primalidad de Rabin y el algoritmo de Ford-Fulkerson para encontrar el flujo óptimo en una red.

La *optimización* es un enfoque matemático para encontrar la mejor solución a un problema dentro de un conjunto de opciones posibles, considerando ciertas limitaciones y objetivos específicos.

Dentro de este enfoque, las *funciones objetivas* son expresiones matemáticas que definen lo que se quiere maximizar (ganancias, eficiencia) o minimizar (costos, desperdicios). Las *variables de decisión* son variables controlables que se ajustan para influir en el resultado. Las *restricciones* son limitaciones que se tienen para alcanzar la optimización.

Se pueden separar por tipo de optimización según las necesidades o estrategias.

Optimización estocástica Trata con incertidumbre o aleatoriedad en la función objetivo y/o las restricciones.	Optimización no lineal Implica funciones no lineales de las variables de decisión.	Optimización sin restricciones No tiene restricciones en las variables de decisión.	Heurística Técnicas para encontrar soluciones aproximadas a problemas complejos cuando encontrar una solución óptima exacta es inviable.
---	--	---	--

Semana 2

Objetivos:

- Investigar y comprender el concepto de PSO como un algoritmo metaheurístico
- Buscar algún ejemplo de PSO para ejecutarlo en Matlab.
- Realizar corrida de escritorio para entender su comportamiento.

Desarrollo:

¿Qué es un algoritmo metaheurístico?

Como ya sabemos, una *heurística* es una técnica utilizada para encontrar soluciones aproximadas a problemas complejos cuando encontrar una solución óptima exacta es inviable,

Las *metaheurísticas* son técnicas de optimización de búsqueda basadas en algoritmos heurísticos que se utilizan para resolver problemas complejos en los que el espacio de búsqueda es muy grande o desconocido. Es muy importante mencionar que no se limitan solo a un problema en específico, sino que se adapta a cualquier problema de optimización dentro de cualquier campo.

Estos algoritmos se basan en la exploración iterativa del espacio de soluciones, tratando de moverse de forma óptima en el espacio de búsqueda y de no caer en óptimos locales.

Se conocen como metaheurísticas las que se encargan de usar estas estrategias, como el algoritmo genético, algoritmo de colonia de hormigas, el recocido simulado, el enjambre de partículas y la optimización por búsqueda tabú, estas son de las más conocidas.

¿Qué es la optimización por enjambre de partículas (PSO)?

La *optimización por enjambre de partículas* es un método de optimización heurística orientado a encontrar mínimos o máximos globales. Su funcionamiento está inspirado en el comportamiento que tienen las bandadas de pájaros o bancos de peces en los que, el movimiento de cada individuo (dirección, velocidad, aceleración, etc.), es el resultado de combinar las decisiones individuales de cada uno con el comportamiento del resto.

Existen diversas variaciones de este algoritmo, con objetivos distintos, pero en términos generales todos siguen una estructura similar para optimizar una función siguen los pasos:

1. Generar un enjambre inicial compuesto por n partículas distribuidas aleatoriamente. Cada una está definida por cuatro elementos.
 - a. Una posición que representa una combinación específica de valores.
 - b. Función objetivo es el valor que resulta de la evaluación de la posición.
 - c. Velocidad es quien determina la velocidad y magnitud del movimiento.
 - d. Registro de la mejor posición alcanzada de cada partícula.
2. Se evalúa cada partícula utilizando la función objetivo.

3. Se actualiza la posición y velocidad de cada partícula, mejorando así la solución de manera progresiva según el número de repeticiones
4. Si no cumple con la condición de detención se repite el proceso desde el segundo paso

Ejemplo de PSO

Para diferentes problemas de optimización, existen diferentes parámetros, pero siguen de base: tamaño del enjambre, número de iteraciones, componentes de velocidad, coeficientes de aceleración.

Los pasos principales del algoritmo son la inicialización, la evaluación de la función objetivo, la iteración y la detención. Siguiendo el proceso:

1. Creación de las partículas iniciales.
2. Se asignan velocidades iniciales a las partículas.
3. En cada posición, se debe evaluar la función objetivo denominada *pBest*, la mejor posición personal.
4. Encontrará el mejor valor y la mejor ubicación más cortos.
5. Las nuevas velocidades actualizadas serán seleccionadas por la simulación, que se basa en las velocidades actuales y las mejores posiciones de las partículas y sus partículas más cercanas.
6. Actualiza repetidamente la ubicación de las partículas.
7. Esta repetición continuará hasta que el algoritmo alcance los criterios de detención.

Para conseguir un código de **PSO**, consulté un artículo de *Simulation Tutor* que incluye un ejemplo de código para **PSO** en **Matlab**.

Recordemos que *Matlab* es un lenguaje de programación de alto nivel que permite el análisis numérico, simulación y visualización de datos, razón por la cual es perfecto para su uso para implementar algoritmos como **PSO**. Fue desarrollado por **MathWorks**. (MathWorks)

Para la demostración en cuestión, el requisito es encontrar un mínimo global, los parámetros a utilizar afectan enormemente el algoritmo:

- Dimensión del problema:
 - Es el número de variables de decisión en el problema de optimización. Cada partícula en el enjambre tiene una posición en un espacio multidimensional, donde cada dimensión representa una variable del problema
- Número de partículas:
 - Se refiere al tamaño del enjambre, es decir, al número total de partículas que exploran el espacio de búsqueda, entre más crece el número, puede mejorar la calidad de la solución encontrada, pero también incrementa el costo computacional.

- **Peso de inercia (inercia):**
 - Es un parámetro que controla la influencia de la velocidad anterior de una partícula sobre su velocidad actual. Ayuda a mantener el equilibrio entre la exploración del espacio de búsqueda y la explotación de las mejores soluciones encontradas.
- **Coefficientes de aceleración:**
 - Son parámetros que determinan la influencia de los componentes cognitivo y social en la actualización de la velocidad de las partículas.
- **Componentes (c_1 y c_2):**
 - Componente cognitivo: Representa la tendencia de una partícula a regresar a su mejor posición individual conocida.
 - Componente social: Representa la tendencia de una partícula a moverse hacia la mejor posición conocida por el enjambre.
- **Influencia:**
 - Los coeficientes de aceleración controlan la velocidad con la que las partículas ajustan su posición basándose en su experiencia individual y colectiva.
- **Número de iteraciones:**
 - Es el número de veces que se actualizan las posiciones y velocidades de las partículas en el enjambre durante la ejecución del algoritmo.
- **Valor aleatorio (rand):**
 - Son valores generados aleatoriamente que se utilizan en los componentes cognitivo y social para agregar aleatoriedad al proceso de búsqueda, evitando el estancamiento en óptimos locales.

Código PSO para Matlab

Función objetivo: $f(x, y) = (x - 20)^2 + (y - 10)^2$, Mínimo/solución: (20,10), para un espacio bidimensional (x, y).

```

5      clear
6      clc
7      iterations = 1000;
8      inertia = 1.0;|
9      correction_factor = 2.0;
10     swarms = 5000;
```

Ilustración Error! No text of specified style in document..1.1

Como muestra la ilustración 1.1, la primera parte se encarga de inicializar las variables importantes, se van a hacer 1000 iteraciones, el factor de inercia se fija en 1, el factor de corrección en 2, y con un total de 5000 partículas.

Inicialización

```

13 % ---- initial swarm position ----
14 swarm=zeros(5000,7);
15 step = 1;
16 for i = 1 : 5000
17     swarm(step, 1:7) = i;
18     step = step + 1;
19 end
20
21 swarm(:, 7) = 1000; % Greater than maximum possible value
22 swarm(:, 5) = 0; % initial velocity
23 swarm(:, 6) = 0; % initial velocity

```

Ilustración 1

Como se muestra en la ilustración 1.2, se crea una matriz de ceros para almacenar las propiedades por cada una de las 5000 partículas del enjambre serán 5000 filas, y 7 columnas para cada propiedad:

swarm=zeros(5000,7);

	Posición en x	Posición en y	Mejor posición en x	Mejor posición en y	Velocidad en x	Velocidad en y	Valor de función
Partículas							

Tabla 1

El ciclo for itera de 1 hasta el número de partículas, en este caso 5000, para actualizar los datos y consultarlos.

Finalmente se inicializa la posición 7 con un valor mayor que el máximo posible para la evaluación de la función (se inicia en 1000), esto para asegurar que cualquier valor calculado de la función objetivo será menor que el valor inicial, y que el valor siempre sea mejor que el anterior.

```

29 %-- position of Swarms ---
30 for i = 1 : swarms
31     swarm(i, 1) = swarm(i, 1) + swarm(i, 5)/1.2 ; %update u position
32     swarm(i, 2) = swarm(i, 2) + swarm(i, 6)/1.2 ; %update v position
33     u = swarm(i, 1);
34     v = swarm(i, 2);
35
36     value = (u - 20)^2 + (v - 10)^2; %Objective function
37
38     if value < swarm(i, 7) % Always True
39         swarm(i, 3) = swarm(i, 1); % update best position of u,
40         swarm(i, 4) = swarm(i, 2); % update best postions of v,
41         swarm(i, 7) = value; % best updated minimum value
42     end
43 end
44

```

Ilustración 2 Posición de las partículas

El código de la ilustración 1.3 calcula las nuevas posiciones de las partículas usando las ecuaciones:

$$u_{nuevo} = u_{actual} + \frac{vel_u}{1.2}, \quad v_{nuevo} = v_{actual} + \frac{vel_v}{1.2}$$

Usa la matriz en la posición correspondiente a u y v (donde $u = x, v = y$) para calcular la nueva posición usando la posición actual más la velocidad correspondiente dividida entre 1.2:

```
swarm (i, 1) = swarm (i, 1) + swarm(i, 5)/1.2 ;
swarm (i, 2) = swarm (i, 2) + swarm(i, 6)/1.2 ;
```

Se guardan las posiciones en u y v para evaluar la función:

```
u = swarm(i, 1);
v = swarm(i, 2);
```

Después usa la función objetivo $f(x, y) = (x - 20)^2 + (y - 10)^2$ para buscar el nuevo mínimo.

```
value = (u - 20)^2 + (v - 10)^2;
```

Se valida si el nuevo valor de la función objetivo obtenido es mejor que el valor anterior de la función (esto por cada partícula).

```
if value < swarm(i, 7)
```

Si el valor calculado de la función objetivo es mejor, significa que las posiciones han mejorado, entonces se actualiza la mejor posición en las columnas correspondientes y el valor obtenido de evaluar la función.

```
swarm(i, 3) = swarm(i, 1);    % update best position of u,
swarm(i, 4) = swarm(i, 2);    % update best positions of v,
swarm(i, 7) = value;          % best updated minimum value
```

Una vez que las 5000 partículas han calculado sus nuevas posiciones, se busca la mejor posición global, es decir, la partícula cuyo valor de la función objetivo se acerque más al mínimo (0).

```
[temp, gbest] = min(swarm(:, 7));    % gbest position
```

El siguiente paso es calcular nuevas velocidades para u y v , e ir acercando el enjambre a la solución.

```
48      %--- updating velocity vectors
49      for i = 1 : swarms
50          swarm(i, 5) = rand*inertia*swarm(i, 5) + correction_factor*rand*(swarm(i, 3)...
51              - swarm(i, 1)) + correction_factor*rand*(swarm(gbest, 3) - swarm(i, 1)); % u velocity parameters
52          swarm(i, 6) = rand*inertia*swarm(i, 6) + correction_factor*rand*(swarm(i, 4)...
53              - swarm(i, 2)) + correction_factor*rand*(swarm(gbest, 4) - swarm(i, 2)); % v velocity parameters
54      end
```

Ilustración 1.3

Para todas las partículas cambia su velocidad (la cual es un factor importante para determinar una nueva posición):

Para u :

$$vel_u = rand * inercia * vel_u + c_1 * rand * (u_{mejor} - u) + c_2 * rand * (u_{mejor\ global} - u)$$

```
warm(i, 5) = rand*inertia*swarm(i, 5) + correction_factor*rand*(swarm(i, 3)...
- swarm(i, 1)) + correction_factor*rand*(swarm(gbest, 3) - swarm(i, 1));
```

Y para v :

$$vel_v = rand * inercia * vel_v + c_1 * rand * (v_{mejor} - v) + c_2 * rand * (v_{mejor\ global} - v)$$

```
swarm(i, 6) = rand*inertia*swarm(i, 6) + correction_factor*rand*(swarm(i, 4)...
- swarm(i, 2)) + correction_factor*rand*(swarm(gbest, 4) - swarm(i, 2));
```

Podemos separar la ecuación en tres términos, la inercia, componente cognitiva y componente social.

Inercia	Componente cognitiva	Componente social
$rand * inercia * vel_u$	$c_1 * rand * (u_{mejor} - u)$	$c_2 * rand * (u_{mejor\ global} - u)$
Este termino calcula la tendencia de una partícula a mantener su velocidad actual	Este termino calcula la tendencia de una partícula a moverse hacia su mejor posición personal	Este termino calcula la tendencia de una partícula a moverse hacia la mejor posición global (la más cercada la solución)

Los valores para los componentes (c_1 y c_2) se fijan en 2 como una medida estándar (como son valores iguales, se usa una variable única `correction_factor`) para mantener un equilibrio en la influencia de las mejores posiciones personales y globales en la actualización de las velocidades de las partículas.

Una vez que cada partícula actualiza las velocidades, se vuelve a calcular las nuevas posiciones, junto con el valor objetivo y se repite el proceso completo hasta que el parámetro que determina el número de iteraciones se detenga. El mínimo puede o no llegar, pero la mejor posición guardará la partícula que más se acerque.

Conclusiones:

Más adelante se realizarán modificaciones a algún algoritmo que ya está establecido con fines de mejorar los resultados, por ahora esta actividad sirve como una introducción y familiarización a los metaheurísticos.

Semana 4 - 5

Objetivos:

- Descargar ***DancingAlgorithm*** (DA) del repositorio en Matlab para su ejecución.
- Realizar corrida de escritorio para comprender su funcionamiento.

Desarrollo:

Introducción

En la búsqueda de soluciones eficientes para problemas de optimización complejos, los algoritmos metaheurísticos han demostrado ser bastante eficientes.

Dentro de este contexto, se presenta a ***DancingAlgorithm*** (DA), es un algoritmo que utiliza una variedad de técnicas inspiradas en el comportamiento de las abejas cuando buscan comida.

A diferencia de *PSO*, DA mejora la exploración del espacio de soluciones mediante estrategias variables y más complejas. Mas concretamente, las técnicas se basan en las danzas de abejas que se encargan de buscar comida:

- Búsqueda global (Waggle dance)
 - Abejas
 - Las abejas exploradoras se alejan del panal o enjambre para ampliar la búsqueda de fuentes de alimento.
 - Partículas
 - Las partículas buscan de manera más general, es decir, se dedican a explorar el espacio de búsqueda para tener más posibilidades de encontrar la solución.
- Búsqueda intermedia
 - Abejas
 - Las abejas comparten información sobre la ubicación de las fuentes de alimento con otras abejas en el panal, quienes luego viajan a las zonas cercanas.
 - Partículas
 - Las partículas utilizan su ubicación anterior para buscar en lugares opuestos.
- Búsqueda local
 - Abejas
 - Las abejas no se alejan demasiado del enjambre, buscan en zonas cercanas.
 - Partículas

- Una partícula busca a las más cercanas y utiliza su posición para mejorar la cobertura en dicha zona.

Cada una de las estrategias que se mencionaron anteriormente tienen ventajas para encontrar la solución y se usan en algunas variantes de *PSO*.

La exploración asociada a la *búsqueda global* incrementa las posibilidades de encontrar la solución. Sin embargo, compromete la convergencia debido a que, si se encuentra un punto prometedor, la exploración continua hacia otras zonas y no explota las zonas cercanas, es más general.

Si la búsqueda se concentra en un solo punto, aumenta la oportunidad de explorar todas posibilidades en esa zona específica. Sin embargo, al incorporar una búsqueda en dirección opuesta no se pasan por alto otras áreas potencialmente prometedoras. No obstante, podría incrementar el uso de recursos.

Cuando en la *búsqueda local*, una partícula tiene una posición privilegiada en la que su posición se acerca bastante a la función objetivo, atrae a otras partículas para incrementar la búsqueda en esa zona específica y aumentar la convergencia del enjambre hacia la solución óptima. Mas, sin embargo, esta es un arma de doble filo, también crecen las posibilidades del enjambre de quedar atrapado en un mínimo local.

Recordemos que las heurísticas son técnicas utilizadas para encontrar soluciones aproximadas a problemas complejos cuando encontrar una solución óptima exacta es inviable. En un problema complejo, las aproximaciones suelen ser más eficientes, entonces puede ocurrir que el enjambre se quede atascado en un mínimo local

Podemos tomar de ejemplo a un alpinista que trata de buscar la cima en una montaña, pero hay varios valles en ella y cada valle cuenta con una elevación (no necesariamente tan alta como la cima), pero hace parecer que es el punto más alto, porque realmente lo es, pero solamente dentro de ese valle.

Y como se favorece la explotación en lugar de la exploración, el enjambre cree que ha llegado a la solución más optima.

DancingAlgorithm implementa estas estrategias de manera aleatoria, es decir, cada partícula en el enjambre utiliza al menos una de las 3 danzas. Esto mejora absolutamente la convergencia hacia la solución. Ya no depende de una sola técnica para cambiar la posición, sino que puede usar 3 diferentes y además esto ocurre de manera aleatoria, esto mantiene un equilibrio entre exploración y explotación.

Código en Matlab

El algoritmo consta de varias funciones importantes que se usan dentro de la función principal con el nombre: *DancingAlgorithm.m*

Inicialización

Primeramente, se manda a llamar la función *test_functions_range* con los valores (*F_index* para seleccionar el problema y *dim* para establecer la dimensión del problema) para devolver el rango de búsqueda y asignarlos a los valores de búsqueda inferior y superior (*Low* y *Up*) que serán valores para mantener el espacio de búsqueda.

```
[Low,up,dim]=test_functions_range(F_index,dim);  
%Initialization  
  
X = initialization(dim,Np,up,low);  
  
fitness = evaluateF(X,F_index);  
  
[fitnessA, indexSorted] = sort(fitness,'ascend');  
  
bestX = X(indexSorted(1),:);  
  
bestFitness = fitnessA(1);  
  
GraphE(1,1) = bestFitness;  
  
probLevy = 0.25;  
  
count = 1;
```

En la ilustración se muestra la inicialización de valores importantes antes de iniciar la búsqueda.

Se inicializa una matriz (*X*) con una población de *Np* partículas, de *dim* dimensiones, y dentro del espacio de búsqueda (definido por *Low* y *Up*):

```
X = initialization(dim,Np,up,low);
```

Después se evalúa la aptitud de cada individuo en la población *X* utilizando la función de evaluación *evaluateF* y almacena los valores en *fitness*:

```
fitness = evaluateF(X, F_index);
```

La función ordena los valores de *fitness* en orden ascendente, *fitnessA* almacena los valores *fitness* de cada partícula mientras *indexSorted* contiene los índices en *X* correspondientes para *fitnessA*.

```
[fitnessA, indexSorted] = sort(fitness,'ascend');
```

Se guarda el valor más óptimo (el más cercano al mínimo de la función) en *fitnessA*, dentro de la variable:

```
bestFitness = fitnessA(1);
```

Como los valores de fitnessA (valor más cercano a la solución) en una posición corresponden a indexSorted (la partícula asociada a ese valor) en la misma posición, significa que indexSorted(1) es la partícula asociada al mejor valor de función, ósea que es la mejor partícula.

```
bestX = X(indexSorted(1),:);
```

Búsqueda:

El algoritmo va a hacer *count* iteraciones

```
while count <= MaxGeneration
```

Para cada una de las partículas

```
for index = 1 : Np
```

Como se mencionó previamente, cada partícula puede escoger alguna de las 3 fases de búsqueda.

Antes de entrar al ciclo que itera a las partículas, se define una variable con un valor aleatorio entre 0 y 1 para definir a que estrategia de búsqueda será escogida.

```
r = rand;
```

Las estrategias de búsqueda se dividen por medio la condicional if, de modo:

```
while count <= MaxGeneration
```

```
for index = 1 : Np
```

```
if (r >= 0.5) <- 50% de probabilidades
```

```
Para la búsqueda global (Waggle dance)
```

```
elseif(r >= 0.3) <- 30% de probabilidades
```

```
Búsqueda intermedia (Sickle dance)
```

```
Else
```

```
Búsqueda local (Local search)
```

```
end <- 20% de probabilidades
```

Después de que cada partícula ha calculado su nueva posición, se usa el mejor fitness de cada una para compararla con el mejor global, en caso de ser mejor, se actualiza. (Parte elitista)

```
if (fitness(index) < bestFitness) %Elitista
```

```
bestX = X(index,:);
```

```
bestFitness = fitness(index);
```

```
end
```

end

Fases de búsqueda

Waggle dance (búsqueda global)

Concepto de salto dentro de algoritmos de optimización

```
if (r >= 0.5) %Waggle dance (Global search)
```

Las partículas exploran de manera global dentro del espacio de búsqueda, para cada partícula se actualiza la posición.

```
X(index,:) + ((bestX - X(index,:)) .* rand(1, dim)) + (LevyFlight(dim,  
X(index,:), bestX));
```

Donde $((\text{bestX} - \text{X}(\text{index},:)) \cdot \text{rand}(1, \text{dim}))$ representa un movimiento hacia el mejor valor bestX , modulado aleatoriamente por $\text{rand}(1, \text{dim})$. Esto es un componente de *búsqueda global* donde una partícula se mueve hacia el mejor conocido (global o personal) con una variabilidad aleatoria.

Y $\text{LevyFlight}(\text{dim}, \text{X}(\text{index},:), \text{bestX})$ aplica un "salto de Levy", que es un tipo de movimiento aleatorio basado en distribuciones estocásticas de Lévy.

Ahora, se asegura de que la nueva posición de la partícula, $\text{X}(\text{index},:)$, esté dentro de los límites definidos por el espacio de búsqueda. La función *findrange* implementa un proceso de recorte que ajusta la posición de la partícula a un rango válido, especificado por los límites low (mínimos) y up (máximos).

```
X(index,:) = findrange(X(index,:),low,up,dim,1);
```

Para la evaluación de la aptitud (fitness) de una partícula en su posición actual $\text{X}(\text{index},:)$ se utiliza la función de evaluación *evaluateF*.

```
fitness(index) = evaluateF(X(index,:), F_index);
```

Donde $\text{fitness}(\text{index})$ es un arreglo que almacena el valor de aptitud de la partícula en la posición especificada por el índice index y la posición de la partícula se obtiene de $\text{X}(\text{index},:)$. El segundo argumento, F_index , es un parámetro que representa el índice de una lista de funciones (problemas o benchmarks) que se va a evaluar con el valor fitness actual.

Middle search

Esta fase está implementando una técnica de reflexión de partículas dentro de los límites del espacio de búsqueda, como una forma de "corregir" las posiciones de las partículas cuando se alejan de los límites definidos. Si la partícula se encuentra fuera de los límites,

se refleja en el espacio de búsqueda. Después de reflejar la partícula, su aptitud se evalúa, y si la nueva posición es mejor (tiene una aptitud menor), se actualiza la posición de la partícula.

```

for j = 1 : dim
    if size(up,2)==1
        xOB(1,j) = low + up - X(index,j);
    end
    if size(up,2) > 1
        high=up(j);down=low(j);
        xOB(1,j) = down + high - X(index,j);
    end
end
xOB = findrange(xOB,low,up,dim,1);
fOB = evaluateF(xOB,F_index);
if fOB < fitness(index)
    X(index,:) = xOB;
    fitness(index) = fOB;
end

```

La fase comienza con un ciclo **for j = 1 : dim**, este corre cada dimensión del espacio del espacio de búsqueda de la partícula (de 1 hasta dim), actualizando la posición de la partícula en cada dimensión si es necesario

Como se ha mencionada anteriormente, esta fase utiliza los límites del espacio de búsqueda para aplicar la técnica de reflexión. Hay dos condiciones a tomar en cuenta:

Una vez dentro del ciclo, **if size(up,2)==1** verifica si up tiene solo una columna, es decir, si todos los límites superiores son iguales. Entonces se realiza una reflexión simple de la posición de la partícula donde la nueva posición reflejada se calcula como:

$$xOB(1,j) = low + up - X(index,j);$$

Aquí, xOB es la solución reflejada (solo es temporal para no modificar la partícula real), low es el límite inferior del espacio de búsqueda, up es el límite superior y X(index, j) es la posición de la partícula en la dimensión j (j es la dimensión actual, se define dentro del ciclo que recorre de j = 1 hasta dim)

`if size(up,2) > 1`: la condición verifica si `up` tiene mas de una columna, es decir, los límites superiores son diferentes para cada dimensión del espacio de búsqueda. Aquí la reflexión se realiza en cada dimensión de forma independiente:

```
high=up(j); down=low(j);
```

Aquí `high` y `down` toman el valor correspondiente para `low` y `up`, dependiendo de la dimensión definida por `j`.

```
xOB(1,j) = down + high - X(index,j);
```

La nueva posible solución es igual a la suma de ambos límites en la `j`-ésima posición y restando la partícula actual dentro de la misma posición `X(index, j)`.

Cuando el ciclo `for` termina, el arreglo `xOB` el reflejo de la partícula actual, entonces se define si se encuentra dentro de los límites del espacio de búsqueda usando la función `findrange`:

```
xOB = findrange(xOB, low, up, dim, 1)
```

La nueva posición reflejada, `xOB`, se evalúa usando la función de aptitud `evaluateF`. El valor resultante, `fOB`, es la aptitud de la solución reflejada:

```
fOB = evaluateF(xOB, F_index);
```

Si la nueva posición reflejada (`xOB`) tiene una mejor aptitud, entonces se actualiza la posición de la partícula a `xOB` y su aptitud a `fOB`.

```
if fOB < fitness(index)
    X(index,:) = xOB;
    fitness(index) = fOB;
end
```

Local search (Búsqueda de vecindad basada en espiral logarítmica)

A diferencia de la búsqueda global, la búsqueda local es un enfoque que se utiliza para encontrar soluciones óptimas dentro de un espacio de búsqueda restringido, partiendo de una solución inicial y explorando el vecindario de soluciones cercanas. Este tipo de algoritmo de búsqueda se basa en la idea de mejorar progresivamente una solución a través de movimientos locales, en lugar de buscar una solución global desde el principio.

En esta fase, se busca alrededor de la partícula actual, buscando las más cercanas para mejorar la aptitud local. De ahí el nombre.

```
d = [];
for i = 1 : Np
```

```

        if i ~= index
            d(i) = Eudistancia(X(index,:),X(i,:));
        end
    end

    [d,indexes] = sort(d,'ascend');

    alpha = -1+count*((-1)/MaxGeneration);

    t = (alpha-1)*rand+1;

    for i = 2 : 3

        rAux = abs((X(index,:) * rand) - X(indexes(i),:));

        X(indexes(i),:) = (rAux .* exp(t).* cos(2*pi*t)) +
        X(indexes(i),:);

        X(indexes(i),:) = findrange(X(indexes(i),:),low,up,dim,1);

        fitness(indexes(i)) = evaluateF(X(indexes(i),:),F_index);

    end

```

Se define una matriz $d = []$; vacía para almacenar las partículas próximas a la actual. Se recorren todas las partículas por medio de un ciclo `for i = 1 : Np` para calcular la distancia euclidiana con respecto a la partícula actual (es una medida que se utiliza para calcular la distancia más corta entre dos puntos en un espacio n-dimensional, siguiendo una línea recta.)

```

for i = 1 : Np
    if i ~= index
        d(i) = Eudistancia(X(index,:),X(i,:));
    end
end

```

Para todas las partículas, mientras no sean la actual (`if i ~= index`), se manda a llamar la función `Eudistancia` para calcular la distancia euclidiana con respecto a la actual y se guarda dentro de la matriz `d(i) = Eudistancia(X(index,:),X(i,:));`

Ahora, la matriz d contiene la distancia euclidiana con respecto a la partícula actual (las distancias calculadas se mantienen en el mismo orden original con sus posiciones), y se ordenan:

```

[d,indexes] = sort(d,'ascend');

```

Sort ordena de manera ascendente las distancias [d,indexes], d contiene las distancias en orden ascendente, indexes indica las posiciones originales de los elementos antes de ser ordenados (para conocer cuál es la posición de dicha partícula).

Esta búsqueda local consiste en centralizar la búsqueda alrededor de la partícula actual, y se usan las dos más cercanas a esta. La búsqueda se realiza alrededor del espacio local, se hace con una espiral logarítmica y atraer a las demás partículas, buscando encontrar un punto óptimo.

```

alpha = -1+count*((-1)/MaxGeneration);
t = (alpha-1)*rand+1;
for i = 2 : 3
    rAux = abs((X(index,:) * rand) -
X(indexes(i),:));

    X(indexes(i),:) = (rAux .* exp(t).*
cos(2*pi*t)) + X(indexes(i),:);

    X(indexes(i),:) =
findrange(X(indexes(i),:),low,up,dim,1);

    fitness(indexes(i)) =
evaluateF(X(indexes(i),:),F_index);

end

```

Elitismo

Finalmente, se evalúa si el fitness obtenido para esa partícula en específico es mejor que la global, en caso de serlo, sería sustituida por la anterior y si no, se mantiene la mejor global.

```

if (fitness(index) < bestFitness)

    bestX = X(index,:);

    bestFitness = fitness(index);

end

```

Conclusiones:

Este algoritmo será utilizado en algunas pruebas en las semanas posteriores y modificado para obtener mejores resultados.

Semana 6 - 7

Objetivos:

- Verificar la función **WeibullFlight** del repositorio en Matlab.
- Para el “DancingAlgorithm” Cambiar la función **WeibullFlight** dentro de **Waggle dance** en lugar de **Levyflight**.

Desarrollo:

Evolución diferencial

La evolución diferencial es un potente algoritmo metaheurístico de optimización evolutiva basado en poblaciones, diseñado para resolver problemas matemáticos complejos en espacios continuos (sin la necesidad de calcular derivadas). Su arquitectura clásica funciona mejorando iterativamente una población de posibles soluciones mediante tres operadores principales (mutación diferencial, cruce, y selección).

- Mutación diferencial: Genera nuevas variaciones escalando la diferencia matemática (vectorial) entre individuos existentes en la población actual.
- Cruce: combina la información de las soluciones actuales con las nuevas variaciones.
- Selección: Decide de manera determinista (basadas en el fitness) qué soluciones sobreviven para la siguiente generación.

Y aunque es muy exitoso, el documento señala que la DE clásica tiene una debilidad: tiende a sufrir de convergencia prematura o estancamiento si los individuos se agrupan demasiado rápido alrededor de un punto que no es la solución óptima real (como un mínimo local), perdiendo así su fuerza de exploración.

¿Qué es el vuelo de Weibull?

Ahora bien, el vuelo de Weibull es un operador estocástico avanzado de perturbación espacial (como una especie de caminata aleatoria) que se inyecta en algoritmos como el antes mencionado (**Evolución diferencial**) para solucionar precisamente sus problemas de estancamiento.

En lugar de mover a los agentes de búsqueda de forma puramente aleatoria o basándose solo en la diferencia entre ellos, el vuelo de Weibull utiliza la distribución de probabilidad de Weibull (mediante funciones como `wblrnd`) para determinar qué tan grande será el paso o salto que dará una solución en el espacio de búsqueda. También equilibra la exploración y explotación al ajustar matemáticamente la forma y escala de esta distribución, el algoritmo puede dar grandes saltos ocasionales (para escapar de zonas engañosas u óptimos locales) y pasos microscópicos controlados

Código en MATLAB

Tomando en cuenta la versión original de este algoritmo tomada del artículo proporcionado por el Dr. Jorge Gálvez, se tiene el Weibull flight:

Inicialización:

```
function Xnew=WeibullFlight(X,Best_agent,dim)

Xnew=X;
```

La función recibe tres parámetros: la posición inicial (**x**), el mejor agente con el mejor fitness encontrado (**Best_agent**), y la dimensión (**dim**). Y entonces se crea una copia del agente actual (**Xnew = x**) esto para no modificar todas las dimensiones del agente, si no solo algunas.

Fase 1 (exploración pura, 25% de probabilidad):

```
if rand <=0.25

    X=X+sign(rand(1,dim)-0.5).*wblrnd(1,1,[1 dim]);
```

De un número aleatorio (**rand**), si es menor o igual a 0.25 el agente ignora al **x_best** y explora por su cuenta, donde se define la dirección y magnitud:

- **Dirección:** **sign(rand(1,dim)-0.5)**, se genera un vector de la misma longitud que las dimensiones, donde cada valor es aleatoriamente +1 o -1. Esto decide si el agente se moverá hacia adelante o hacia atrás en cada eje.
- **Magnitud:** **wblrnd(1,1,[1 dim])**, genera el tamaño del salto usando una distribución de Weibull en escala 1 y forma 1. Permite dar saltos de tamaño variable, incluyendo saltos largos ocasionales para escapar de mínimos locales.

Fase 2 (explotación, 75% de probabilidad):

Si el número aleatorio es mayor a 0.25, el agente intenta aprovechar la información de la mejor solución conocida, en donde hay dos casos posibles:

Caso A:

```
else

    if isequal (Best_agent,X)

        step=0.1*sign(rand(1,dim)-0.5);

        X=X+step.*wblrnd(0.5,1,[1 dim]);
```

Si **X** y **Best_agent** son exactamente iguales, significa que este agente está en la mejor posición descubierta.

En lugar de dar un salto grande que lo alejaría de esta buena zona, calcula un paso muy pequeño ($\text{step} = 0.1$). Luego, realiza una búsqueda local multiplicando ese paso pequeño por un número aleatorio de Weibull (con una escala reducida de 0.5). Esto ayuda a refinar la solución y encontrar el mínimo local.

Caso B:

```
else
    step=0.5*sign(rand(1,dim)-0.5)*norm(Best_agent-X);
    X=X+step.*wblrnd(0.5,1,[1 dim]); %0.5
end
end
```

Si el agente no es el mejor, entonces se acerca a él. Calcula la distancia exacta entre su posición y la del mejor usando la norma euclidiana `norm(Best_agent, X)`.

El tamaño del **step** base es proporcional a esa distancia (multiplicada por 0.5 y por una dirección aleatoria).

Y finalmente multiplica este paso por el ruido de Weibull. Esto significa que el agente se acerca hacia el mejor, pero dando zigzags probabilísticos para no perderse nada en el camino.

Fase 3 (fase de cruce y actualización):

```
id=randi(dim);
ind=find(rand(1,dim)<=1.5/dim);
if isempty(ind)
    ind=[ind, id];
end
Xnew(ind)=X(ind);
end
```

Esta última parte decide qué posición del agente original se van a actualizar con la nueva posición calculada.

Se genera un vector aleatorio y se seleccionan aquellos índices donde el valor es mejor o igual a $1.5/\text{dim}$. Esto significa que, en promedio se actualizarán alrededor de 1.5 dimensiones en cada iteración, sin importar cuán grande sea el espacio de búsqueda.

Se hace uso de `if isempty(ind)` en caso de que no se seleccione ninguna dimensión, el código fuerza la inclusión de al menos una dimensión al azar (`ind`).

Y finalmente, `Xnew(ind)=X(ind)` inyecta los nuevos valores calculados **solo** en las dimensiones seleccionadas, devolviendo el agente resultante.

Ejemplo Vuelo de Weibull en 2 dimensiones

Condiciones iniciales

Para un problema de 2 dimensiones (`dim = 2`)

- Posición del agente actual: `[10, 10]`
- Posición del mejor Agente (`Best_agent`): `[0, 0]`

Ejecución:

<code>Xnew = X;</code> • Se crea la copia de seguridad. • <code>Xnew</code> ahora vale <code>[10, 10]</code>.
<code>if rand <= 0.25</code> • Se genera un número aleatorio entre 0 y 1. • Suponiendo que se genera 0.85. • Como 0.85 no es menor a 0.25, se cambia al <code>else</code>.
<code>if isequal(Best_agent, X)</code> • Se evalúa si <code>[0, 0]</code> es exactamente igual a <code>[10, 10]</code>, pero es Falso. • Entonces se salta al <code>else</code> para entrar en la fase de Atracción.
<code>step = 0.5 * sign(rand(1,dim)-0.5) * norm(Best_agent-X);</code> Aquí matemáticamente sucede: • Distancia (norm): Calcula la distancia de <code>[10, 10]</code> a <code>[0, 0]</code>. La fórmula es ◦ $\sqrt{(0 - 10)^2 + (0 - 10)^2} = \sqrt{200} \approx 14.14$ • Dirección aleatoria: <code>rand(1,2)</code> genera dos números, suponiendo <code>[0.2, 0.9]</code>. ◦ Al restarles 0.5 quedan: <code>[-0.3, 0.4]</code>. ◦ La función <code>sign</code> saca el signo: <code>[-1, 1]</code>. (Significa: muévete a la izquierda en X, y hacia arriba en Y). • Cálculo final del <code>step</code>: <code>0.5 * [-1, 1] * 14.14 = [-7.07, 7.07]</code>. ◦ El <code>step</code> base está listo.
<code>X = X + step .* wblrnd(0.5, 1, [1 dim]);</code>

• Ruido de Weibull: La función <code>wblrnd</code> genera 2 números aleatorios basados en la distribución de cola larga. Supongamos que escupe <code>[0.3, 0.8]</code>. • Multiplicación: <code>[-7.07, 7.07] .* [0.3, 0.8] = [-2.12, 5.65]</code>. (Este es nuestro salto real modificado por Weibull). • Suma final: <code>X = [10, 10] + [-2.12, 5.65] = [7.88, 15.65]</code>. ◦ El agente calculó su posición deseada. Está más cerca en X, pero se desvió en Y explorando.
El cruce <code>id = randi(dim);</code> → Genera un número entero aleatorio (1 o 2). Supongamos que <code>1. ind = find(rand(1,dim) <= 1.5/dim);</code> • Umbral: $1.5 / 2 = 0.75$. • <code>rand(1,2)</code> genera, digamos, <code>[0.9, 0.4]</code>. • Se compara: $0.9 \leq 0.75$ (Falso), $0.4 \leq 0.75$ (Verdadero). • Por lo tanto, <code>ind</code> guarda solo el índice que cumplió la condición: <code>ind = [2]</code>.
if isempty(ind) • <code>ind</code> contiene 2, entonces se ignora este caso.
Xnew(ind) = X(ind); • <code>Xnew</code> era <code>[10, 10]</code>. • Se toma la posición 2 de la candidata X (15.65) y se reescribe en la coordenada 2 de <code>Xnew</code>. Resultado final: Xnew = [10, 15.65].

Aplicación dentro de “Waggle dance”

El objetivo es usar `WeibullFlight` para sustituir el salto de Levy como parámetro para la ecuación, para de esta manera obtener mejores resultados en la búsqueda global:

Tenemos esta ecuación donde las partículas exploran de manera global dentro del espacio de búsqueda. Aquí es donde se modifica el salto de Levy por el salto de Weibull.

```
X(index,:) + ((bestX - X(index,:)) .* rand(1, dim)) + (LevyFlight(dim,
X(index,:), bestX));
```

Solo se pone el nombre de la función (ya fue generada dentro de la carpeta del proyecto) como parámetro:

```
X X(index,:) = X(index,:) + ((bestX-X(index,:)) .* rand(1,dim)) +
(WeibullFlight(X(index,:),bestX, dim));
```


Resultados

Para poder comparar los resultados de la versión original (en donde **Levy flight** es usada) y la versión con el **Weibull flight**, se tomarán ciertos parámetros como partida (Sphere y Rastrigin son por lo general funciones muy básicas y simples que sirven como buen punto de partida para la evaluación de algoritmos, mientras que 2 y 30 son de las mejores dimensiones para estas funciones), esto se repetirá 100 veces para cada versión, donde:

- La función de prueba sea **Sphere** ($F_index = 1$), 50 partículas, 1000 iteraciones y 2/30 dimensiones.

	Levy flight	Weibull flight	Levy flight	Weibull flight
100 veces →	$F_index = 1$; $N_p = 50$; $MaxGeneration = 1000$; $dim = 2$;	$F_index = 1$; $N_p = 50$; $MaxGeneration = 1000$; $dim = 2$;	$F_index = 1$; $N_p = 50$; $MaxGeneration = 1000$; $dim = 30$;	$F_index = 1$; $N_p = 50$; $MaxGeneration = 1000$; $dim = 30$;
Resultado →	Mejor Fitness Encontrado : 1.588434e-238 (Corrida 17) Peor Fitness Encontrado : 4.575710e-200 Fitness Promedio: 5.242346e-202	Mejor Fitness Encontrado : 1.468922e-05 (Corrida 10) Peor Fitness Encontrado : 5.217910e-02 Fitness promedio: 7.399577e-03	Mejor Fitness Encontrado : 5.793149e-121 (Corrida 32) Peor Fitness Encontrado : 1.984622e-107 Fitness promedio: 2.049293e-109	Mejor Fitness Encontrado : 9.209262e+01 (Corrida 58) Peor Fitness Encontrado : 5.187410e+03 Fitness promedio: 1.876939e+03
Los resultados obtenidos no mostraron una mejoría en comparación a como estaba originalmente cuando el vuelo de Levy estaba como parámetro. El fitness promedio obtenido en ambos casos (para 2 y 30 dimensiones) sigue demostrando que la versión original sigue siendo mejor.				

- La función de prueba sea **Rastrigin** ($F_index = 10$), 50 partículas, 1000 iteraciones y 2/30 dimensiones.

	Levy flight	Weibull flight	Levy flight	Weibull flight
100 veces →	$F_index = 10$; $N_p = 50$; $MaxGeneration = 1000$; $dim = 2$;	$F_index = 10$; $N_p = 50$; $MaxGeneration = 1000$; $dim = 2$;	$F_index = 10$; $N_p = 50$; $MaxGeneration = 1000$; $dim = 30$;	$F_index = 10$; $N_p = 50$; $MaxGeneration = 1000$; $dim = 30$;
Resultado →	Mejor Fitness Encontrado : 0.000000e+00 (Corrida 1)	Mejor Fitness Encontrado : 2.634591e-05	Mejor Fitness Encontrado : 0.000000e+00 (Corrida 1)	Mejor Fitness Encontrado : 3.495610e+01 (Corrida 80)

	Peor Fitness Encontrado : 0.000000e+00 Fitness Promedio: 0.000000e+00	(Corrída 4) Peor Fitness Encontrado : 3.161768e-02 Fitness Promedio: 5.404408e-03	Peor Fitness Encontrado : 0.000000e+00 Fitness Promedio: 0.000000e+00	Peor Fitness Encontrado : 2.289207e+02 Fitness Promedio: 1.386066e+02
Los resultados obtenidos no mostraron una mejoría en comparación a como estaba originalmente cuando el vuelo de Levy estaba como parámetro. El fitness promedio obtenido en ambos casos (para 2 y 30 dimensiones) sigue demostrando que la versión original sigue siendo mejor.				

Conclusiones:

Los resultados obtenidos no fueron mejores que la versión original, para posteriores modificaciones, se busca cambiar alguna de las fases de búsqueda para ver si mejoran los resultados.

Semana 8 - 9

Objetivos:

- Abordar la lectura de las primeras 7 páginas del PDF proporcionado sobre el artículo “The Tangent Search Algorithm for Solving Optimization Problems”.
- Generar un reporte del funcionamiento del TSA.

Desarrollo:

Introducción

El artículo *The Tangent Search Algorithm for Solving Optimization Problems* establece que la optimización es un campo fundamental en disciplinas como la ciencia, la ingeniería, la medicina, la biología y la química. Su objetivo principal es encontrar los mejores valores para las variables de decisión con el fin de maximizar el rendimiento operativo o minimizar los costos asociados. Sin embargo, muchos problemas de optimización del mundo real presentan características matemáticas y topológicas desafiantes, tales como espacios de búsqueda no convexos, costos computacionales prohibitivos y la masiva presencia de múltiples óptimos locales. Estas trabas estructurales vuelven impracticables su resolución sistemática mediante métodos matemáticos exactos o basados puramente en derivadas. Por ello, las metaheurísticas han emergido como una alternativa aproximada, poderosa y sumamente flexible para abordar estos desafíos, proporcionando soluciones de alta calidad en tiempos de cómputo razonables de forma global.

En este contexto de innovación analítica, se desarrolló el Algoritmo de Búsqueda de Tangente (TSA, por sus siglas en inglés), un novedoso algoritmo de optimización de naturaleza estocástica y basado en poblaciones. Este enfoque utiliza un modelo matemático riguroso inspirado en el comportamiento asintótico y periódico de la función trigonométrica tangente para guiar el "vuelo" y la búsqueda de soluciones a través del espacio de búsqueda. A diferencia de otras metaheurísticas inspiradas en procesos biológicos o naturales que requieren de decodificaciones complejas, el TSA se destaca por su extrema simplicidad estructural, su alta eficiencia computacional y la ventaja técnica de requerir un número muy reducido de parámetros configurables por el usuario.

La arquitectura geométrica del algoritmo logra mantener un balance ininterrumpido y dinámico entre la exploración (búsqueda expansiva de nuevas áreas) y la explotación (intensificación microscópica del espacio de búsqueda). Además, el modelo integra un novedoso mecanismo probabilístico de escape (Escape Local mínima) diseñado explícitamente para evitar que las partículas queden atrapadas irremediablemente en óptimos locales engañosos. Esto funciona en perfecta simbiosis con un tamaño de paso adaptativo y variable (variable step-size) modulado de forma no lineal por una función logarítmica. Esta progresión geométrica permite ajustar dinámicamente la intensidad de las búsquedas: despliega pasos amplios en las iteraciones

tempranas para cruzar valles profundos, e involuciona a movimientos microscópicos en las etapas finales, mejorando contundentemente la convergencia fina hacia las soluciones óptimas absolutas.

Conceptualización del algoritmo TSA

Como ya se ha mencionado anteriormente, este algoritmo únicamente requiere de un número muy reducido de parámetros definidos por el usuario. Esto con el fin de evitar la sobrecarga de ajuste paramétrico.

Motivación teórica para el uso del enfoque tangente

El algoritmo de búsqueda tangente basa su comportamiento y eficiencia en una simple función matemática, la función tangente. ¿Por qué esta función?, ¿Qué es lo que ofrece este tipo de funciones que puede mejorar este tipo de algoritmos?

La función tangente presenta dos propiedades que ayudan enormemente a mantener un equilibrio entre exploración y explotación.

- Variación entre $-\infty$ y $+\infty$:
La función tangente no está acotada y toma todos los valores reales. A medida que el ángulo se acerca a valores donde coseno es igual a cero, la tangente tiende a $-\infty$ o $+\infty$ creando asíntotas verticales.
- Periodicidad: Es una función periódica con periodo $\pi(180)$, lo que significa que $\tan \theta + \pi = \tan \theta$, esto contrasta el seno y el coseno, cuyo periodo es 2π .

Ahora bien, todas las ecuaciones que componen al algoritmo TSA tiene con parámetro un paso global: $step * \tan(\theta)$, es regido por la función tangente por las propiedades ya antes mencionadas.

Modelo matemático del algoritmo

Es importante mencionar que la mayoría de los algoritmos de optimización se basan fundamentalmente en un esquema de descenso:

$$X^{t+1} = X^t + step * d$$

Ecuación 1

Donde α representa el tamaño del paso (*step size*) y d es la dirección del movimiento. La distinción principal entre los diversos métodos radica en la forma de calcular dicho paso. Mientras que los métodos basados en derivadas emplean información del Gradiente o de la matriz *Hessiana*, los métodos de derivada libre utilizan pasos estocásticos para converger hacia el óptimo global.

La eficacia de un algoritmo depende de la calidad de su paso:

- Valores elevados fomentan la exploración del espacio de búsqueda.

- Valores pequeños favorecen la explotación de las zonas prometedoras.

Es por eso que se propone un nuevo mecanismo de desplazamiento basado en la función tangente, al cual se denomina "*Tangent flight*" (vuelo tangencial). Como se observa en la *Figura 1*, la mayor distribución de puntos se concentra alrededor del ángulo 0, lo que potencia la explotación. No obstante, la presencia de puntos distantes y aislados permite una exploración efectiva del espacio.

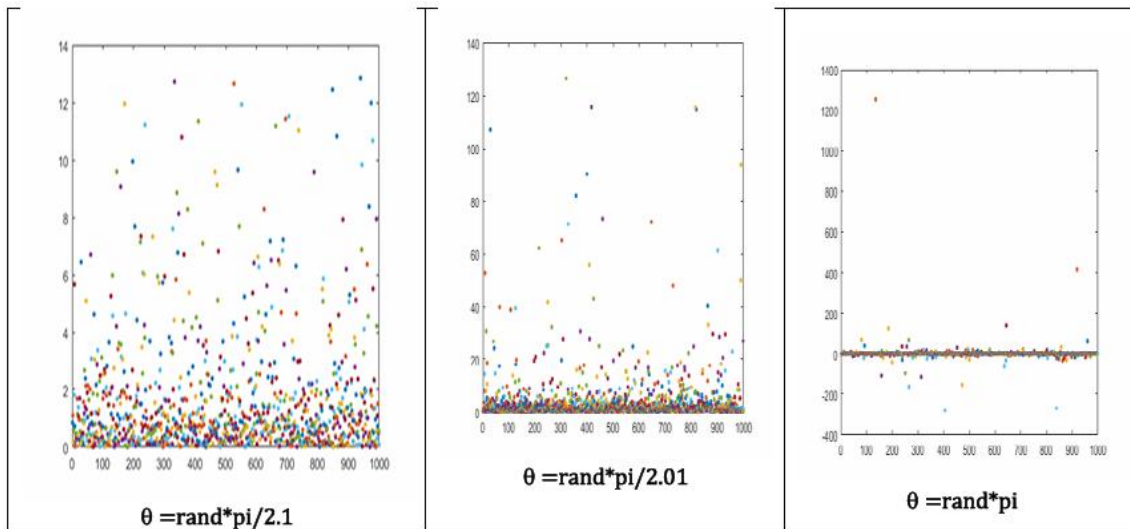


Figura 1 Comportamiento de la función tangente con diferentes rangos de ángulo.

Para gestionar esta alta capacidad exploratoria y asegurar una convergencia rápida, el *Tangent flight* se multiplica por una función decreciente. En la *Figura 2* se ilustra el comportamiento de este mecanismo al interactuar con una función logarítmica decreciente: la búsqueda comienza con una exploración amplia en las primeras etapas y se refina gradualmente hacia la explotación conforme avanzan las iteraciones.

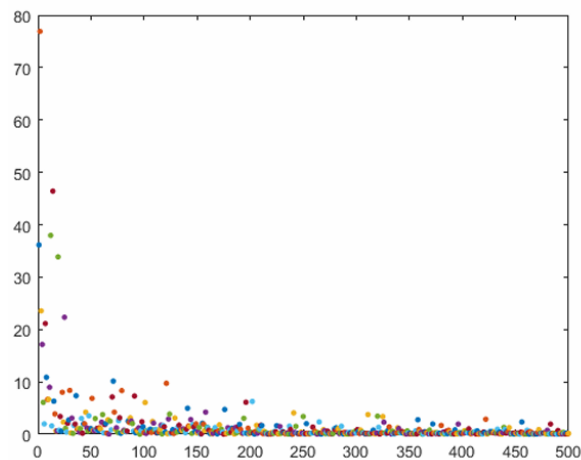


Figura 2 Comportamiento de la función tangente con un step decreciente.

Descripción del algoritmo

Todo buen algoritmo de optimización busca tener un balance entre explotación y exploración. Se tiene bastante claro que el continuo uso de la explotación deriva en que los algoritmos tengan más probabilidad de quedar atrapados en mínimos locales, lo que daría malos resultados para la maximización o minimización de resultados. En caso contrario, el excesivo uso de la exploración trae como consecuencia que los algoritmos sean demasiado lentos y diverjan de una solución óptima.

Teniendo estos inconvenientes en cuenta, el propósito de la fase de exploración es buscar y encontrar los mejores candidatos a dar una solución óptima manteniendo un cierto equilibrio entre los dos tipos de búsqueda. En tanto que el escape de mínima local se aplica de manera aleatoria a algún agente (posible solución) para evitar que quede atrapado en un mínimo local. Como se puede apreciar en la *figura 3*.

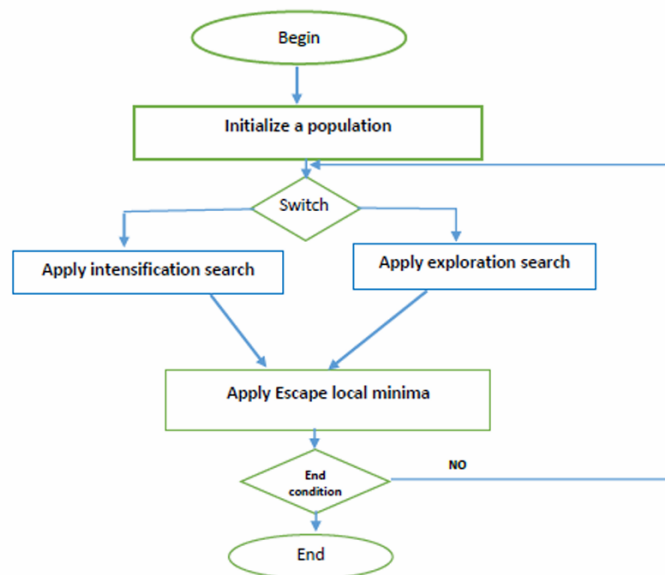


Figura 3 Diagrama de flujo del algoritmo TSA.

Inicialización

El TSA inicia su ciclo vital computacional generando aleatoriamente una población inicial poblada de agentes búsqueda que es sembrada especialmente y confinada estrictamente dentro de las fronteras que delimitan el espacio de búsqueda para el problema abordado.

Para evitar sesgos topológicos iniciales, la matriz de soluciones se distribuye probabilísticamente de manera uniforme a través del hiperespacio de búsqueda. La

posición posicional de cada agente en cada dimensión es computada formalmente mediante la siguiente ecuación matricial generatriz:

$$X0 = lb + (ub - lb) \cdot \text{rand}(D);$$

Ecuación 2

Aquí *lb* y *ub* representan los límites paramétricos inferiores (*lower bounds*) y superiores (*upper bounds*) del problema dimensional evaluado; la función *rand()* actúa como un generador de variables estocásticas, teniendo números aleatorios distribuidos de manera aleatoria dentro del rango continuo; y la variable *D* denota la magnitud escalar de la dimensión topológica del problema de optimización.

Búsqueda de Intensificación

La fase de intensificación está diseñada para que el TSA ejecute, en primera instancia, un paseo local guiado y restringido. Esta micro - exploración se rige paramétricamente por la introducción de un vector de atracción hacia el óptimo histórico. Matemáticamente se describe como:

$$X_i^{t+1} = X_i^t + \text{step} * \tan(\theta) * (X_i^t - \text{opt}S_i^t)$$

Ecuación 3

En esta ecuación el término $(X_i^t - \text{opt}S_i^t)$ representa el diferencial espacial o la distancia vectorial euclidiana entre la población actual del agente X_i y la posición de la mejor solución óptima global encontrada por la población hasta ese momento temporal $\text{opt}S_i^t$. La función $\tan(\theta)$ modula la agresividad de este acercamiento.

Posterior a esta perturbación posicional, la fase de intensificación aplica un mecanismo de asimilación paramétrica directa. Algunas variables específicas de la solución recién perturbada son sobrescritas y reemplazadas forzosamente por los valores escalares exactos de la variable dimensional correspondiente almacenada en la solución óptima actual $\text{opt}S$. Este proceso de clonación parcial se rige por:

$$X_i^{t+1} = \text{opt}S_i^t$$

*Ecuación 4. En caso de que la variable *i* sea seleccionada.*

La arquitectura del TSA impone reglas estrictas sobre el volumen de este intercambio genético paramétrico. La proporción total de variables que sufren este reemplazo es equivalente al 20% del total de dimensiones si el problema posee una dimensionalidad geométrica superior a 4 variables ($D > 4$). Por el contrario, para problemas de baja dimensionalidad (con 4 o menos variables espaciales), esta tasa de asimilación aumenta agresivamente hasta el 50%.

En consecuencia analítica, la nueva solución candidata estructurada X_i^{t+1} mantiene intencionalmente una proporción de similitud biunívoca con la mejor solución actual que nunca excede el techo límite del 50%. Esta restricción previene el colapso de diversidad genética poblacional, permitiendo a la vez que la solución actual sea mejorada y refinada localmente mediante la inyección del material de mayor calidad disponible en el enjambre.

Para prevenir que estas manipulaciones topológicas violentas expulsen al agente fuera del dominio viable del problema, cada solución resultante X es sometida incondicionalmente a una rutina de reparación de contención espacial. Si algún valor escalar se desborda paramétricamente de los límites lb y ub , es reubicado estocásticamente dentro del espacio válido mediante:

$$X(X < lb) = rand * (ub - lb) + lb;$$

$$X(X > ub) = rand * (ub - lb) + lb;$$

Ecuación 5

Asegurando de esta forma la total viabilidad operativa de la población.

Búsqueda de Exploración

La fase de exploración se constituye del producto entre un tamaño de paso (*step*) dinámicamente variable y la función de vuelo de tangente. Como se estableció teóricamente, cuando el ángulo estocástico θ se evalúa en magnitudes cercanas a $\frac{\pi}{2}$, la evaluación trigonométrica de la tangente explotará asintóticamente, arrojando valores escalares gigantescos. Este fenómeno empujará cinemáticamente a la solución obtenida a una distancia topológica masiva respecto a la solución inicial. Contrariamente, cuando θ decae hacia valores cercanos a 0, la tangente entrega magnitudes diminutas, y el vector resultante ubicará a la nueva solución en una vecindad extremadamente íntima a la solución actual.

Por consiguiente, la ecuación cinemática que rige la búsqueda de exploración actúa como un mecanismo híbrido que fusiona topológicamente los recorridos aleatorios globales y locales dentro de un único operador sin fisuras. La ecuación de exploración se define vectorialmente como:

$$X_i^{t+1} = X_i^t + step * \tan(\theta)$$

Ecuación 6

La alteración espacial no se aplica de manera masiva e indiscriminada a todo el vector solución. En su lugar, el **TSA** impone una disciplina de evaluación probabilística donde esta alteración tangencial es aplicada de manera independiente a cada dimensión o variable escalar del agente con una tasa estricta

de probabilidad igual a $1/D$, donde D representa la dimensión paramétrica total del problema de optimización.

Mecanismo de escape (escape de mínima local)

Para evadir categóricamente el problema de estancamiento en mínimos locales, el TSA incorpora en su ciclo principal un mecanismo procedimental específico y novedoso, diseñado exclusivamente para lidiar de forma correctiva con agentes paralizados.

Este procedimiento de escape se invoca y evalúa en cada iteración global basándose en una probabilidad de ejecución maestra denominada P_{esc} . El mecanismo opera en dos fases de decisión condicional anidadas.

En cada iteración temporal, si se cumple el umbral del P_{esc} , exactamente un agente de búsqueda es secuestrado y seleccionado completamente al azar de entre toda la población demográfica (X). A este agente seleccionado se le induce un vector escalar temporal denominado R , el cual decae logarítmicamente con el tiempo para calibrar la severidad del escape:

$$R = 10 * \text{sign} / \log(1+t);$$

Ecuación 7

A continuación, el algoritmo evalúa un factor probabilístico crítico (un 99% de las veces, el agente entra en la primera rama del flujo, mientras que un 1% de las veces sufre una erradicación completa). Dentro del 99% de ejecuciones regulares, el agente tiene un 80% de probabilidad de sufrir una dislocación gravitacional dirigida violentamente por un diferencial relativo a la solución óptima absoluta actual ($OPTX$):

$$X = X + R * (\text{optS} - \text{rand} * (\text{optS} - X)) ;$$

Ecuación 8

Si no cae en ese 80% (es decir, el 20% restante del bloque principal), el agente experimenta un salto ciego lateral masivo determinado únicamente por la fuerza bruta de la función tangente y las fronteras absolutas del problema (ub , lb):

$$X = X + \tan(\text{teta}) * (ub - lb);$$

Ecuación 9

Seudocódigo del proceso de escape de mínima local:

```

Input solution X
generar step  $R = 10 * \text{sign} / \log(1+t)$ ;
if rand  $\leq 0.99$ 
If rand  $< 0.8$ 

```

```

    X = X + R.*(OPTX- rand*(OPTX-X)) ;
else
    X = X+ tan(teta)*(ub-lb);
End
Else
Reemplazar X por una solución aleatoria generada por la ecuación 1.
end
Output nueva solución X

```

Arquitectura general del TSA (seudocódigo)

La orquestación general y la toma de decisiones algorítmicas entre ejecutar la fase de Intensificación o la fase de Exploración no se realiza secuencialmente, sino que se dictamina en tiempo real durante cada iteración evaluando un parámetro estocástico global denominado *Pswitch*. Para cada agente en la población, se genera un número aleatorio uniforme; si este valor es menor a *Pswitch*, el algoritmo ejecuta el conjunto de ecuaciones de Intensificación; en caso contrario, activa el mecanismo de Exploración.

```

While t <= MAX_FE // Máximo de las funciones de evaluación

For each agente buscar Xi (i=1:T, donde T es el tamaño de la población)

If rand < Pswitch

    Aplicar búsqueda de intensificación;

Else

    Aplicar búsqueda de exploración;

End

End for each
If rand < Pesc
    Se selecciona un agente de manera aleatoria (G);
    Aplicar escape de mínimo local (G);

    t=t+1;
End while

```

Conclusiones:

Como futuro objetivo se plantea integrar este algoritmo en la fase de búsqueda local de **Dancing Algorithm (DA)**, con el fin de optimizar su rendimiento y aportar un mayor dinamismo en la exploración de soluciones óptimas.

Semana 10

Objetivos:

- Cambiar la variable de variancia de baile
- Cambiar las ecuaciones de búsqueda local por las ecuaciones 3 y 4 del Tangent Search Algorithm.

Desarrollo:

Introducción

Los resultados obtenidos al evaluar **Dancing Algorithm** con las funciones de prueba *Sphere* y *Rastrigin* posterior al cambio realizado en una de las ecuaciones de la búsqueda global (*Waggle dance*):

Levy flight:

```
X(index,:) = X(index,:) + ((bestX-X(index,:)) .* rand(1,dim)) +  
(LevyFlight(dim,X(index,:),bestX));
```

Weibull flight:

```
X(index,:) = X(index,:) + ((bestX-X(index,:)) .* rand(1,dim)) +  
(WeibullFlight(X(index,:),bestX, dim));
```

No resultó en la mejoría de las soluciones en comparación a los obtenidos con la versión de Levy flight:

	Sphere (2 dimensiones)	Rastrigin (2 dimensiones)	Sphere (30 dimensiones)	Rastrigin (30 dimensiones)
Levy flight	Fitness Promedio: 5.242346e-202	Fitness promedio: 0.000000e+00	Fitness promedio: 2.049293e-109	Fitness promedio: 0.000000e+00
Weibull flight	Fitness promedio: 7.399577e-03	Fitness Promedio: 5.404408e-03	Fitness promedio: 1.876939e+03	Fitness Promedio: 1.386066e+02

Por consiguiente, se propone el cambio en las condiciones de entrada para la selección de bailes en el algoritmo (variable de variancia de baile para selección de algún tipo de búsqueda), además de la sustitución total de la búsqueda local (*vecindad basada en espiral logarítmica*) por las ecuaciones 3 y 4 del algoritmo de búsqueda tangente (**TSA**) y comprobar si existe alguna mejoría en comparación con los resultados obtenidos con la versión original del algoritmo.

Recapitulación

Como se mencionó con anterioridad, la búsqueda de intensificación realiza una micro exploración geométrica restringida, en donde se utiliza la función tangente para acercarse estocásticamente a los agentes hacia la mejor solución actual del enjambre.; además, aplica un cruce directo donde el 20% (o 50% en bajas dimensiones) de las variables de la nueva solución son reemplazadas por las de la mejor solución histórica, asegurando refinamiento sin perder diversidad.

Es importante mencionar que en la ecuación 3 se hace uso de un parámetro que no he explicado antes, el *step* (este “paso” en específico está diseñado para la intensificación, es por ello el “paso” que se utiliza en la exploración no nos interesa):

$$\text{step1} = 10 * \text{sign}(\text{rand} - 0.5) * \text{norm}(\text{optS}) * \log(1 + 10 * \text{dim} / t)$$

Ecuación 10

Y para calcular el tamaño de paso variable. Se utiliza un alternador direccional (*sign*), la norma euclidiana de la mejor solución actual (*norm(bestX)*), y lo multiplica por una función logarítmica descendente ($\log(1 + 10 * \text{dim} / FE)$). Este logaritmo asegura que los pasos sean grandes al inicio de las iteraciones. (FE es una variable que irá incrementado en 1 para cada iteración de cada partícula de modo que conforme avanzan las iteraciones los pasos se ajustan en función de la proximidad con la mejor solución global).

Conversión a código

Ecuación 3:

$$X_i^{t+1} = X_i^t + \text{step} * \tan(\theta) * (X_i^t - \text{optS}_i^t)$$

Ecuación 3

Definición de parámetros:

- La posición actual $\rightarrow X_i^t$
- El paso $\rightarrow \text{step}$
- La tangente del ángulo aleatorio $\rightarrow \tan \theta$
- La solución optima $\rightarrow \text{optS}_i^t$

El agente actual ya existe al igual que la mejor solución obtenida, **x(index,:)** y **bestX** respectivamente. Entonces hace falta el paso y el ángulo:

Para el ángulo(*teta*):

$$\text{teta} = \text{rand} * \text{pi} / 2.1;$$

Es importante mencionar que el ángulo se multiplica por $(\pi / 2.1)$ para mantenerlo estrictamente por debajo de $\frac{\pi}{2.1}$ (90 grados) por que la función tangente tiende a infinito en $\frac{\pi}{2}$. Al usar 2.1 en el divisor, se evita que el algoritmo dé saltos infinitamente grandes que arruinen la búsqueda.

Para el step:

```
step = sign(rand - 0.5) * norm(bestX) * log(1 + 10 * dim / FE);
```

El step determina qué tan lejos se moverá la solución.

$sign(rand - 0.5)$ decide aleatoriamente si el paso será hacia adelante (positivo) o hacia atrás (negativo); $norm(bestX)$ saca la norma euclidiana en razón de la mejor solución encontrada para funcionar como dirección;

finalmente $log(1 + 10 * dim / FE)$ es función logarítmica descendente que se encarga de ir disminuyendo los pasos a medida que las iteraciones avanzan.

Antes de armar la ecuación con los parámetros, hay algo importante que se debe de tomar en cuenta, en caso de que una solución actual sea exactamente igual a la mejor global encontrada (o en el caso de que varias soluciones sean iguales y se encuentren en la mejor posición global), esta ecuación prácticamente no haría nada porque la dirección sería nula (ya están en la misma posición) y se perdería una iteración para encontrar a mejor solución. Para evitar esto, se sugiere que en caso de que la solución actual es exactamente igual a la mejor encontrada, se multipliquen todos los elementos de la solución actual por un número aleatorio (un escalar) para que la diferencia con la mejor solución pueda arrojar algo y continuar con la búsqueda de una mejor solución.

```
if isequal(bestX, X(index,:))  
    X(index,:) = X(index,:) + step * tan(teta) .* (rand .* X(index,:) - bestX);  
else  
    X(index,:) = X(index,:) + step * tan(teta) .* (X(index,:) - bestX);  
End
```

Ecuación 4:

$$X_i^{t+1} = optS_i^t$$

Ecuación 4. En caso de que la variable i sea seleccionada.

Una vez que la ecuación 3 ha hecho una exploración y se ha movido en dirección a la mejor solución global, la función de la ecuación 4 escoge de manera aleatoria un porcentaje de las variables de la solución actual (20% para problemas con una dimensión mayor que 4, y 50% para problemas con una dimensión menor o igual que 4) y se cambian por las de la mejor encontrada con el objetivo de refinar lo que ya se tiene para ver si una pequeña variación mejora el resultado.

Con esto en cuenta:

Se define esa condición:

```
if dim > 4
    pb = 0.2;
else
    pb = 0.5;
end
```

Y se crea una variable para que escoja de manera aleatoria (dependiendo de la dimensión del problema) las dimensiones:

```
ind = find(rand(1, dim) <= pb);
```

En caso de que por el número aleatorio no es escoja ninguna dimensión, es forzar a que se vuelva a hacer una nueva elección:

```
if isempty(ind)
    ind = randi(dim);
end
```

Finalmente, se hace el cambio de las dimensiones para la solución actual y se incrementa la variable FE para la siguiente iteración:

```
X(index,ind) = bestX(ind);
FE = FE + 1;
```

Se ajustan los rangos del límite en caso de que sea superado:

```
X(index,:) = findrange(X(index,:), low, up, dim, 1);
```

Y se manda a evaluar la solución para comprobar el resultado y posteriormente determinar si es apto para ser la mejor solución global

```
fitness(index) = evaluateF(X(index,:),F_index);
```

Implementación en DA (código completo)

Para la implementación en **DA**, se definen dos variables importantes en la cabecera junto a las demás inicializaciones, $FE=1$ (para el logaritmo en el paso) y $Pb=0$ (para la probabilidad en las dimensiones). Debido a que dim es una constante, no es necesario estar comprobando a cuál caso pertenece (20% o 50%), entonces lo coloco fuera del ciclo *for* que itera a las partículas y lo mantengo dentro del bucle principal (**while count <= MaxGeneration**).

Posteriormente, en *Local search*:

```
teta = rand * pi / 2.1;
step = sign(rand - 0.5) * norm(bestX) * log(1 + 10 * dim / FE);
if isequal(bestX, X(index,:))
    X(index,:) = X(index,:) + step * tan(teta) .* (rand .* X(index,:) - bestX);
else
    X(index,:) = X(index,:) + step * tan(teta) .* (X(index,:) - bestX);
end
ind = find(rand(1, dim) <= pb);
if isempty(ind)
    ind = randi(dim);
end
X(index,ind) = bestX(ind);
X(index,:) = findrange(X(index,:), low, up, dim, 1);
fitness(index) = evaluateF(X(index,:),F_index);
FE = FE + 1;
```

Conclusiones:

Tangent search Algorithm es un algoritmo poderoso, y que a partir de las demostraciones en el artículo se comprueba su eficiencia y eficacia. Es por ello por lo que una implementación de sus ecuaciones dentro de DA podría traer consigo mejoras en los resultados antes obtenidos con las funciones de prueba. Para esta actividad de la semana en específico solo se planteó la implementación de las ecuaciones 3 y 4 de intensificación con el objetivo de mejorar *Local search*. Por lo que las demás ecuaciones fueron ignoradas, incluyendo el escape de mínima local, aunque es una de las mayores innovaciones que tiene este algoritmo; sería una estupenda implementación en un futuro.

Semana 11 - 12

Objetivos:

- Comparar resultados del algoritmo original y el modificado.
- Hacer pruebas especiales.

Desarrollo:

Comparación de resultados

Como en las pruebas realizadas con el cambio del Weibull flight no fueron satisfactorias, la intención ahora es comparar los resultados que obtienen al evaluar el algoritmo con la adición del TSA, y comparar los resultados con la versión original.

Para esta evaluación se hará uso de las funciones Sphere y Rastrigin con 2 y 30 dimensiones para ambas versiones del algoritmo.

Dancing Algorithm (original)

Sphere:

100 veces →	F_index = 1; Np = 50; MaxGeneration = 1000; dim = 2;	F_index = 1; Np = 50; MaxGeneration = 1000; dim = 30;
Resultado →	Mejor Fitness Encontrado : 1.588434e-238 (Corrida 17) Peor Fitness Encontrado : 4.575710e-200 Fitness Promedio: 5.242346e-202	Mejor Fitness Encontrado : 5.793149e-121 (Corrida 32) Peor Fitness Encontrado : 1.984622e-107 Fitness promedio: 2.049293e-109

Rastrigin:

100 veces →	F_index = 10; Np = 50; MaxGeneration = 1000; dim = 2;	F_index = 10; Np = 50; MaxGeneration = 1000; dim = 30;
Resultado →	Mejor Fitness Encontrado : 0.000000e+00 (Corrida 1) Peor Fitness Encontrado : 0.000000e+00	Mejor Fitness Encontrado : 0.000000e+00 (Corrida 1) Peor Fitness Encontrado : 0.000000e+00 Fitness Promedio:

	Fitness Promedio: 0.000000e+00	0.000000e+00
--	--	--------------

Dancing Algorithm (con TSA)

Sphere:

100 veces →	F_index = 1; Np = 50; MaxGeneration = 1000; dim = 2;	F_index = 1; Np = 50; MaxGeneration = 1000; dim = 30;
Resultado →	Mejor Fitness Encontrado : 3.634770e-23 (Corrida 45) Peor Fitness Encontrado: 2.297453e-03 Fitness Promedio: 2.740353e-04	Mejor Fitness Encontrado: 7.189603e-02 (Corrida 97) Peor Fitness Encontrado: 2.392596e+00 Fitness Promedio: 8.041953e-01

Rastrigin:

100 veces →	F_index = 10; Np = 50; MaxGeneration = 1000; dim = 2;	F_index = 10; Np = 50; MaxGeneration = 1000; dim = 30;
Resultado →	Mejor Fitness Encontrado: 0.000000e+00 (Corrida 100) Peor Fitness Encontrado: 3.979831e+00 Fitness Promedio: 5.136605e-01 Desviación Estándar: 6.596832e-01	Mejor Fitness Encontrado: 1.233070e+00 (Corrida 6) Peor Fitness Encontrado: 7.434663e+01 Fitness Promedio: 3.459369e+01 Desviación Estándar: 2.218497e+01

Las soluciones que se obtuvieron para ambas funciones reflejaron una mejoría en comparación a las que se obtuvieron al usar Weibull flight, pero se mantienen por debajo de los obtenidos por el algoritmo original.

Evaluación con benchmarks y condiciones especiales.

Este tipo de evaluaciones que se hicieron con las funciones Sphere y Rastrigin no son suficientes para determinar que tan eficiente es un algoritmo, por lo que se va a evaluar con las funciones de prueba (benchmarks) proporcionadas.

Evaluaciones para DA original

Parámetros:

- Np: 100
- Iteraciones: 300,000
- Dimensiones: 30

	-50 50	-100 100
F1	4951125514	2606718.288
F2	4.51E+03	4.66E+03
F3	1007.62351	932.267079
F4	1900	1900
F5	1910658.2	2.47E+07
F6	2.20E+03	2036.165448
F7	1267182.01	168132.747
F8	6007.69448	2314.66462
F9	3348.90558	3253.05939
F10	2985.16026	2926.29154

Evaluaciones para DA (con TSA)

Parámetros:

- Np: 100
- Iteraciones: 300,000
- Dimensiones: 30

	-50 50	-100 100
F1	3971780538	3886659.05
F2	4808.0046	4599.31316
F3	898.094385	871.564027
F4	1900	1900
F5	1900.2959	1900.2158
F6	2109725.69	1142608.66
F7	2116.31894	2076.23413

F8	1283110.71	88118.9264
F9	2980.03212	2314.1297
F10	3144.30663	3106.47434

Conclusiones:

Aún con las modificaciones en la fase de búsqueda local, no hubo una mejoría, de hecho, los resultados de la versión original son absolutamente mejores. Se queda a la espera de las siguientes indicaciones.

Semana 13 - 14

Objetivos:

- Visualizar y entender el algoritmo de optimización COC (Cluster -based optimization with chaotic behavior).
- Realizar corrida de escritorio para comprobar su funcionamiento.

Desarrollo:

Introducción

Siguiendo la línea de los algoritmos metaheurísticos, el algoritmo **Cluster Based Optimization With Chaotic Behavior** también está diseñado para encontrar el valor mínimo de una función tal como ya se ha demostrado con otros algoritmos como el PSO y el Dancing Algorithm. Nota importante: (El algoritmo es demasiado complejo y está formado por funciones externas y dependencias que no son necesarias de explicar a fines de las instrucciones dadas y para el reporte)

Algo que es importante destacar es que este algoritmo tiene un funcionamiento que se aleja un poco del concepto clásico de enjambre o poblaciones. Su estrategia principal consiste en dividir a la población de posibles soluciones en grupos para explorar el espacio de búsqueda de manera local y global.

Hay algunos conceptos fundamentales que son necesarios explicar (por los cuales se rige el COC):

- **Clustering o agrupamiento:** En lugar de que todos los individuos se muevan a ciegas hacia la mejor global y la población se divide en subgrupos. Cada subgrupo tiene un líder (el mejor del clúster).
- **Comportamiento Caótico:** Utiliza vectores generados por mapas caóticos para calcular el tamaño y dirección de los pasos de los individuos. Esto introduce aleatoriedad controlada, superior a la simple generación de números pseudoaleatorios.
- **Densidad del Clúster:** El movimiento de los individuos dentro de un clúster se ve afectado por qué tan "denso" o compacto es dicho clúster.

Funciones y archivos para considerar

Afuera de la función principal se encuentran algunas dependencias que son realmente importantes para que el archivo principal (COC.m) pueda acceder a las funciones necesarias para realizar las evaluaciones:

- **Centroide.m :**
Calcula el centro geométrico de cada clúster. : Recorre cada cluster formado y calcula el promedio (mean) de las coordenadas de todos los individuos que pertenecen a él. Esto le da al algoritmo un punto de referencia para calcular la densidad.
- **chaosVector.m :**
Es un bloque con opciones que funge como mapas caóticos , que son funciones matemáticas que producen secuencias probabilísticas deterministas. En lugar de pasos aleatorios, las partículas usan estos vectores caóticos para definir la magnitud de su movimiento. El caos ayuda a que el algoritmo no se quede atrapado en óptimos locales
- **clusterDensit.m :**
Se calcula una métrica de densidad para cada *clúster*. Calcula las distancias de los individuos hacia el centroide (*minDist*, *maxDist*), y la fórmula final de densidad está fuertemente basada en la cantidad de miembros que tiene el clúster respecto a una constante. Actúa como un ponderador. Un clúster con ciertas características de agrupación hará que sus miembros den saltos más grandes o más pequeños hacia el líder del clúster.
- **findrange.m :**
Asegura que ningún individuo se salga del espacio de búsqueda permitido revisando cada dimensión de cada partícula. Si una coordenada es menor que el límite inferior (*low*), la iguala a *low*; si supera el límite superior (*up*), la iguala a *up*. También incrementa un contador global.
- **vTransform.m :**
Genera un vector de transformación que modula las búsquedas laterales agregando un comportamiento oscilatorio a la búsqueda. Al usar el coseno, el tamaño del paso puede variar suavemente, permitiendo que las exploraciones locales a la izquierda y derecha del individuo sean más dinámicas.

Función principal

En consideración de la nota anterior, para fines de este reporte solo es necesario explicar la función principal, y esta se encuentra dentro un bucle que se rige por la variable *count* hasta llegar a *MaxGeneration* (número de evaluaciones hasta el límite predefinido), dentro de este ciclo se encuentra el funcionamiento principal.

Inicialización:

- En primera instancia, se crea una población inicial X de manera aleatoria dentro de los límites definidos (*low*, *up*).
- Se evalúa la aptitud de cada partícula con la función *evaluateF*.
- Se ordena la población de mejor a peor y se guarda la mejor solución global (*xbest*)
- Se calcula un factor de escala inicial *alfTotal* basado en el tamaño de búsqueda para que el algoritmo reduzca gradualmente su radio de búsqueda conforme avance.

Agrupamiento jerárquico:

- *Linkage*: Esta función se encarga de agrupar las partículas que ya están ordenadas mediante la distancia euclidiana.
- Corte dinámico (*cutmean*): Calcula un punto de corte basado en la inconsistencia del árbol jerárquico para decidir cuántos clústeres formar en esa iteración.
- Centroides y densidad: Calcula el centro geométrico de cada clúster formado y evalúa su densidad espacial.

Búsqueda y evolución:

El algoritmo realiza dos tipos de refinamiento por cada clúster formado:

Movimientos de los miembros (Intra – Clúster):

- Se identifica a mejor individuo del clúster (*bestCluster*).
- Las demás partículas del clúster se mueven hacia ese líder local; la distancia del salto está influenciada por la densidad de clúster y un vector caótico.
- Refinamiento local: Para cada miembro se realiza una búsqueda hacia la izquierda o derecha (crea un radio de búsqueda mediante *vTransform*); si alguna de esas nuevas posiciones es mejor, el individuo se mueve en esa dirección.

Movimientos de los líderes (Inter-Clúster):

- El líder de cada clúster (*bestCluster*) intenta acercarse al mejor individuo global de toda la población (*xbest*), con un salto de tamaño caótico.
- El líder también realiza una búsqueda rápida a su izquierda y derecha para refinar su posición global.

Actualización (elitismo):

- **Evaluación continua:** Cada vez que un individuo se mueve, se evalúa su nueva posición (*evaluateF*) y se actualiza el historial de convergencia (*fT*). El contador de evaluaciones (*count*) se incrementa.
- **Elitismo:** Al final del bucle, la población se vuelve a ordenar. Si el nuevo mejor individuo es superior al *xbest* global, este último se actualiza.
- **Transición de Etapa:** Si se superan ciertas evaluaciones (*count* > *IT(phase)*), el algoritmo entra a la siguiente fase, reduciendo drásticamente el tamaño del paso (*alfAdaptive*) para pasar de una fase de "exploración" a una de "explotación" fina.

Ejemplo práctico:

Para entender de manera sencilla el funcionamiento de COC, decidí usar parámetros muy básicos con el fin de que sea más fácil de desglosar.

Suponiendo que se tiene a función Sphere, el mínimo global es (0, 0) con un costo de 0. Dimensión de 2, un límite de entre (-10 a 10) y una población de 4.

Fase de Inicialización:

Se lanzan a los 4 individuos al azar en el espacio y evalúan qué tan buenos son:

Suponiendo que el azar (*initialization*) da estos 4 puntos:

- Partícula 1:
(2, 2), Fitness: 8
- Partícula 2:
(-3, 1), Fitness: 10
- Partícula 3:
(5, 5), Fitness: 50
- Partícula 4:
(-1, -1), Fitness: 2

Ordenamiento: El algoritmo ordena de mejor a peor:

1. $X_1 = (-1, -1)$ (xbest)
2. $X_2 = (2, 2)$
3. $X_3 = (-3, 1)$

4. $X_4 = (5, 5)$

Agrupamiento Jerárquico (Clustering)

El algoritmo usa la distancia entre ellos para agruparlos. Suponiendo que el algoritmo de agrupamiento (*linkage de Ward*) decide dividirlos en 2 clústeres por su cercanía en el plano cartesiano:

- **Cluster 1:** Contiene a X_1 y X_3 .
 - **Líder:** X_1 (porque tiene mejor fitness).
 - **Centroide:** Se calcula el punto medio: $(-2, 0)$.
 - **Densidad:** La función *clusterDensity* le asignará un peso pequeño (por ejemplo 0.08) basado en que solo hay 4 individuos en total.
- **Clúster 2:** Contiene a X_2 y X_4
 - **Líder:** X_2 .
 - **Centroide:** Punto medio $(3.5, 3.5)$.
 - **Densidad:** Suponiendo 0.08.

Evolución y Movimientos

El algoritmo procesa un clúster a la vez.

Clúster 2:

El Clúster 2 tiene al miembro X_4 y su líder X_2

Movimiento Intra-Cluster:

La partícula X_4 se da cuenta de que no es el mejor de su grupo, así que decide acercarse a su líder X_2 .

La fórmula simplificada es: $Nueva_Pos = Pos_Actual + (Lider - Pos_Actual) * Caos * Densidad$

- Distancia para cubrir: Líder $(2, 2)$ - Actual $(5,5)$ = Vector $(-3, -3)$.
- Suponiendo que *chaosVector* devuelve 0.5 y la densidad es 0.08.
- Movimiento = $(-3, -3) * 0.5 * 0.08 = (-0.12, -0.12)$.

- Nueva posición de X_4 : $\$(5, 5) + (-0.12, -0.12) = (4.88, 4.88)$.
 - Mejoró un poco.

Refinamiento Local:

Desde su nueva posición (4.88, 4.88), X_4 usa *vTransform* para dar un pasito oscilatorio a su izquierda y a su derecha. Si descubre que la izquierda (4.5, 4.5) es mejor, actualiza su posición allí. Si al calcular estos pasos se sale del rango de -10 a 10, la función *findrange* lo detiene a en la frontera.

Movimiento Inter-Clúster:

Ahora el líder del Clúster 2, X_2 se mueve hacia el mejor global histórico x_{best} (X_1).

La fórmula sería: $Lider_Nuevo = Lider_Actual + (Mejor_Global - Lider_Actual) * transformación * Caos$

- Se acerca hacia (-1,-1) con un salto influenciado por chaosVector. Suponiendo que da un salto fuerte y termina en (0.5, 0.5).
- Se evalúa. El fitness es 0.5 (mejoró considerablemente).
- Hace su propio refinamiento de izquierda/derecha.

Actualización y Elitismo

Se hace este mismo proceso para todos los clústeres. Al terminar la ronda, el algoritmo tiene una nueva población de individuos que se han movido.

Los vuelve a ordenar y evalúa.

- El x_{best} inicial era (-1,-1) con fitness de 2.
- Pero en esta iteración, el líder del Clúster 2 X_2 logró llegar a (0.5, 0.5) con un fitness de 0.5.

Condición de Elitismo:

El mejor de esta iteración es mejor que el histórico ($0.5 < 2$).

El algoritmo declara a **(0.5, 0.5)** como el nuevo x_{best} .

El contador de iteraciones sube 1, se ajusta el parámetro adaptativo para que en la siguiente ronda los saltos sean ligeramente más pequeños y precisos, y el ciclo vuelve a empezar.

Semana 15 - 16

Objetivos:

- Transcribir la función principal (**COC.m**) al lenguaje de programación Python.
- Comprobar resultados con las funciones sphere y Rastrigin.

Desarrollo:

Introducción

El archivo principal del algoritmo **COC.m** hace uso de otras dependencias para generar las funciones y valores que se necesitan las fases del algoritmo, es por ello que para es importante mencionar que para la traducción del algoritmo se simulaban estas condiciones en archivos reducidos y por ende no fue la misma exactitud que en Matlab, pero la traducción es a fines de esta actividad en específico.

Se crearon dos archivos para simular dichas condiciones, *test_functions* y *test_functions_range*.

Como se explicó en ejemplo sencillo (aunque de manera resumida), para la traducción del algoritmo principal a Python se explicará por etapas. También es importante aclarar que no todo el algoritmo podrá ser explicado por este medio, son más de 300 líneas y redactar todo serían muchas páginas, pero se explicará de manera detallada las funciones más importantes.

Inicialización y Evaluación (*initialization, evaluateF*):

Crea una población aleatoria de *np* individuos dentro de los límites definidos (espacio de búsqueda de dimensión *dim* y evalúa el *fitness*.

```
def initialization(dim, Np, up, low)
```

```
def evaluateF(X, F_index):
```

```
def findrange(solution, low, up, dim, type_val)
```

Configuración inicial y cálculo de *alfTotal*:

```
cutmean = 1.0
```

```
(cálculos de rangos)
```

```
alfTotal = total_range / dim
```

Inicialización

```
X = initialization(dim, Np, up, low)
```

```
fitness = evaluateF(X, F_index)
```

Ordenar

```
Index = np.argsort(fitness)

# ... (refinamiento inicial) ...

fT[0] = LightnBest[0]
```

Agrupamiento Jerárquico (Clustering):

En cada generación, agrupa a los individuos según su similitud espacial utilizando el método de *Ward* y la *distancia euclidiana*. Esto divide a la población en distintos nichos o subgrupos.

Se utiliza directamente la librería importada *scipy.cluster.hierarchy*.

Se crea un ciclo while count <= MaxGeneration:

Clustering

```
Z = hierarchy.linkage(totalOrdenado, method='ward',
metric='euclidean')

# Calcular el corte (cutoff)

try:

    I = hierarchy.inconsistent(Z)

    # ... (cálculos de cutmean) ...

except Exception as e:

    cutmean = cutAux if 'cutAux' in locals()
    else 1.0

# Obtener los clusters

clust = hierarchy.fcluster(Z, cutmean,
criterion='distance')

aux_clust = np.max(clust)
```

Cálculo de Densidad y Centroides:

Identifica el centro físico de cada clúster y qué tan dispersos (*densos*) están sus miembros. Sea crean dos funciones

```
def centroide(totalOrdenado, clust):  
  
    def clusterDensity(totalOrdenado, centroid, clust,  
aux_clust, dim):
```

Todo dentro de COC para obtener los clústeres:

Cálculo de Centroide y Densidad

```
centroid = centroide(totalOrdenado, clust)  
  
density = clusterDensity(totalOrdenado, centroid, clust,  
aux_clust, dim)  
  
density_sorted_indices = np.argsort(density)
```

Evolución y Movimiento (Fase de Exploración):

Por cada clúster, los individuos se mueven hacia el mejor individuo de ese clúster. El tamaño de este salto está modulado por la densidad del clúster y un vector.

Se crea la función:

```
def chaosVector(X, low, up, dim, type_c):
```

Al inicio del bucle `for k in range(1, aux_clust + 1):` se valoran a los miembros que no son el líder (`index = f_indices[1:]`).

Se mueven los individuos hacia el líder del clúster

```
if index.size > 0:  
  
    for i in range(index.shape[0]):  
  
        idx = index[i]  
  
        xAux = totalOrdenado[idx, :].copy()  
  
  
        k_density = density[k-1]  
  
        move_vec = (bestCluster - xAux) *  
chaosVector(xAux, low, up, dim, 3) * (k_density)  
  
        totalOrdenado[idx, :] = xAux + move_vec
```

```

totalOrdenado[idx, :] =
findrange(totalOrdenado[idx, :], low, up, dim, 1)

```

Se evalúan y actualiza el resto del conteo.

Refinamiento Local (Fase de Explotación):

Aplica perturbaciones hacia la izquierda y derecha tanto a los individuos regulares como al líder del clúster usando un factor adaptativo (*alfAdaptive*). Si la nueva posición es mejor, el individuo se actualiza.

Se crean dos funciones:

```

def vTransform(dim, alfAdaptive):

def minim(totalOrdenado, F_index, fT, count): (Para
asegurar que el fitness histórico no empeore tras los
movimientos).

```

Dentro de COC: Son tres bloques distintos dentro del bucle de clústeres.

Refinamiento de individuos:

```

vT = vTransform(dim, alfAdaptive)

# ... cálculo de left y right ...

fitnessLR = evaluateF(np.vstack([left,
right]), F_index)

```

Movimiento del líder general (bestCluster):

```

vT = vTransform(dim, alfAdaptive)

chaos_vec_12 = chaosVector(bestCluster, low,
up, dim, 12)

bestCluster = bestCluster + (xbest -
bestCluster) * vT * chaos_vec_12

```

Refinamiento del líder:

```

vT = vTransform(dim, alfAdaptive) left =
bestCluster - (bestCluster * chaos_vec_12) *
vT- bestCluster) * vT * chaos_vec_12

```

Control de Fases y elitismo:

El algoritmo divide sus generaciones en tres fases de esfuerzo (70%, 20%, 10%). A medida que avanza de fase, el paso de búsqueda espacial (*alfAdaptive*) se reduce drásticamente, permitiendo que el algoritmo pase de explorar todo el mapa a afinar la mejor solución encontrada.

Se crea una función:

```
def evolutionProcess(EP, MaxGeneration):
```

Dentro de COC: La llamada inicial a *evolutionProcess* y el bloque al final del ciclo while.

En la configuración inicial

```
EP = [70, 20, 10]
```

```
IT = evolutionProcess(EP, MaxGeneration)
```

En la parte de elitismo y ordenamiento

```
IndexBest = np.argsort(LightnBest)
```

```
# ... (Elitismo) ...
```

```
if LightnBest[0] < fobj:
```

```
    xbest = totalOrdenado[0, :].copy()
```

```
    fobj = LightnBest[0]
```

Actualización de la fase de evolución

```
if phase < len(IT) - 1:
```

```
    if count > IT[phase]:
```

```
        phase += 1
```

```
        div = div * 100
```

Resultados

Al igual que con todas las pruebas que se han hecho con los otros algoritmos, se creó un archivo en específico para poder ejecutar el algoritmo con mas control sobre los parámetros y no tener que depender de la consola.

Las condiciones de ejecución son de tal manera, que para cada n numero de partículas, hay x iteraciones y los resultados respectivamente, esto con el fin de ver el comportamiento de ambos algoritmos en determinadas condiciones

Sphere:

Versión original:

Dim: 2						
for i in range 2000						
MaxIteraciones						
		10	20	30	100	200
Np	10	2.39E+03	2400.481	1241.646	335.3979	129.5125
		0	0	0	0	0
	20	740.4686	783.5679	791.4454	187.7833	1.752854
		0	0	0	0	0
	30	294.4246	294.9344	317.2992	38.26665	0.691291
		0	0	0	0	0
	40	113.864	120.0307	105.8804	113.6399	7.494234
		0	0	0	0	0
	50	54.05433	47.07847	46.41502	55.389	0.53375
		0	0	0	0	0
	60	28.61869	20.39241	18.0075	20.79803	0.052179
		0	0	0	0	0
	70	10.19755	9.922139	9.120717	9.709476	8.948232
		0	0	0	0	0

Versión de Python:

Dim: 2						
for i in range 2000						
MaxIteraciones						
		10	20	30	100	200
Np	10	4.61E+03	4630.1981	0.530688	0	0
		136.74472	3039.1444	0	0	0
	20	2432.984	2345.4616	2452.176	0	0
		1999.2554	1274.6076	996.5388	0	0
	30	1610.0117	1596.5145	1633.819	0	0
		312.429	586.30809	2740.424	0	0
	40	1260.9816	1254.7424	1240.235	1222.515	0
		896.58741	105.99796	768.6633	1312.911	0
	50	973.48123	1001.6418	1010.242	964.5616	0
		1.0142848	867.66342	102.6237	820.5447	0
	60	824.68112	864.89561	847.2369	854.5785	0
		256.50626	2012.3807	331.1983	356.6225	0
	70	716.82325	735.55493	760.7324	725.0296	753.4687
		114.99282	238.99432	111.5643	225.58	989.8202

Rastrigin:

Versión original:

for i in range 2000						
MaxIteraciones						
		10	20	30	100	200
Np	10	7.18E+00	7.079389	2.949264	0.767931	0.055461
		0.011003	0.003717	0.00013	0	0
	20	4.633475	4.604878	4.559381	1.965012	0.219437
		0.002631	0.006226	0.004255	0.000205	0
	30	3.609498	3.616985	3.512393	0.928116	0.342065
		0.00219	0.000458	0.000979	0	0
	40	2.982793	2.935435	2.965626	2.966633	1.118405
		0.000504	0.00194	0.003876	0.00056	0.000107
	50	2.543313	2.645697	2.600281	2.596681	0.514
		0.000346	0.000684	0.000968	0.000094	0
	60	2.312435	2.368548	2.374961	2.275873	0.422672
		0.000415	0.00071	0.000464	0.000431	0
	70	2.097744	2.120967	2.217121	2.136133	2.117832
		0.00104	0.000284	0.000845	0.000014	0.000121

Versión de Python:

for i in range 2000						
MaxIteraciones						
		10	20	30	100	200
Np	10	1.59E+01	15.746113	8.609431	3.509368	1.78162
		17.191804	20.645905	4.412532	10.25113	10.49815
	20	11.608067	11.596674	11.72927	5.351753	1.971972
		6.536917	12.742263	7.484335	6.611738	1.030644
	30	9.497822	9.623663	9.896515	4.319022	0.0074
		13.602265	11.898227	5.72514	3.144019	0
	40	8.570527	8.528839	8.444836	8.523606	3.680116
		9.5110909	6.0966798	7.309887	6.989632	2.468347
	50	7.681841	7.578992	7.608446	7.562005	3.155452
		8.896319	0.6352445	5.155214	8.894744	7.580238
	60	7.001988	7.050026	7.034729	6.941782	2.786718
		4.4211406	13.057585	4.269481	10.35104	6.393918
	70	6.583573	6.518557	6.535729	6.671265	6.563041
		7.2630555	4.5034031	7.510503	8.58586	1.11538

Sphere + Rastrigin:

Versión original:

for i in range 2000						
MaxIteraciones						
Np		10	20	30	100	200
	10	2.53E+03	2423.105	1161.127	382.7914	151.9002
		0	0	0	0	0
	20	719.7001	747.7377	768.1883	146.7223	4.983514
		0	0	0	0	0
	30	258.7134	297.1692	311.789	30.21124	7.228498
		0	0	0	0	0
	40	137.7981	113.6235	126.8851	123.0174	6.501704
		0	0	0	0	0
	50	55.61425	48.72104	47.04666	48.8028	0.442652
		0	0	0	0	0
	60	24.20446	25.30039	20.37033	25.17432	0.024423
		0	0	0	0	0
	70	11.13087	8.310303	10.54933	9.960897	12.73892
		0	0	0	0	0

Versión en Python:

for i in range 2000						
MaxIteraciones						
Np		10	20	30	100	200
	10	1.65E+02	166.22661	108.5499	70.25278	54.13949
		16.239439	21.347253	144.6763	2.417549	18.21149
	20	52.167265	52.698436	52.07182	33.4305	22.45204
		32.452575	9.5281894	8.810158	15.93864	12.11731
	30	29.173076	29.552024	28.03474	19.15007	14.95191
		63.738747	12.680488	31.00276	81.28988	2.027141
	40	19.268542	19.806469	20.25926	19.81148	12.72848
		15.023631	0.2053959	3.990297	4.666828	12.96231
	50	15.774463	15.518176	15.18286	15.09959	10.57754
		50.021769	1.0041765	19.6272	14.79604	9.936276
	60	12.327717	12.545921	12.52089	13.19596	8.269264
		15.966656	3.9799029	7.82399	7.496396	6.149946
	70	10.411577	10.8392	10.68358	0	11.04145
		29.218684	2.4696703	48.47728	0	4.78503

Conclusiones:

Hubo buenos resultados, el comportamiento que se tiene con las condiciones son buenos, pero no mejores que la versión original. Se espera que con una correcta implementación de las dependencias necesarias.